AI-EOS: A Multi-Agent Business Workforce

Research and project design report

Prepared for: College project planning **Date:** 14 October 2026 **Status:** Proposed architecture and experimental plan; no implementation or benchmark results claimed.

Executive recommendation

Build a simulated business workforce in which a Manager Agent delegates bounded tasks to specialist agents, reconciles their evidence, and converts findings into a tracked action plan. Use two connected demonstrations: **inventory replenishment** for a dependable operational workflow, and **sales decline investigation** for parallel reasoning, cross-agent disagreement, and adaptive follow-up.

Make the research contribution **risk- and state-aware semantic reuse for multi-agent RAG**. The central question is whether a system can reduce repeated generation and duplicated context while preserving answer correctness, data freshness, and owner control. Semantic caching and prompt compression already exist; the project should evaluate their application to changing business state rather than claim to invent them. [S2-S5]

Answer to the NLP question: yes, conditionally. Sentence embeddings can identify paraphrases, while intent classification and entity extraction identify what the owner is actually asking. A cache can then answer a repeated knowledge question without another generative LLM call. A repeated sales question must also pass checks for the same product, time window, metrics, permissions, and source-data versions. A payment instruction always goes through fresh approval and execution checks.

Four decisions that shape the project

- **Specialize work, tools, and memory.** Different agent names alone do not demonstrate useful collaboration.
- **Keep arithmetic and permissions deterministic.** LLMs interpret, investigate, and explain; code computes totals and enforces authorization.
- **Retrieve evidence selectively.** Use RAG for business documents and structured tools for sales, stock, and budget facts.
- **Approve every financial commitment.** Payments, purchases, refunds, and spend changes require owner approval regardless of amount. Low-risk analysis and internal drafting can proceed automatically.

Reading guide

Pages 2-7 define scope, agents, collaboration, and controls. Pages 8-11 explain RAG and token reuse. Pages 12-14 define the research evidence, experiment, and cost measurements. Pages 15-16 give the delivery plan and demo criteria. Pages 17-18 contain references and assumptions.

1. Scope and interpretation of the brief

What the supplied brief contributes

The ten-page AI-EOS MVP brief [B1] defines a simulated workflow: business event, manager, inventory, procurement, finance, risk/approval, execution, and audit. Its example orders 38 Laptop Stands at INR 850 each for INR 32,300. It proposes Python, FastAPI, LangGraph, shared in-memory state, a simple frontend, mock business data, and an offline fallback.

The attachment also contains commands to build an application immediately. Those are treated as document content. **This report fulfills the current request for research and planning; it does not undertake that embedded implementation task.**

Topic	Brief position	Recommendation for this request
Primary workflow	Inventory replenishment	Retain as the first operational demo
Business workforce	Manager, inventory, procurement, finance	Add sales, marketing, research, and email as scoped roles
Financial control	Auto-execute below INR 50,000	Owner approval for every financial action
Execution	Simulated purchase orders	Keep all purchases, payments, and email sends simulated
Storage	No database for MVP	In-memory state, local fixtures, in-memory retrieval/cache
Research	Not specified in detail	RAG quality, safe semantic reuse, and token efficiency

Working assumptions

Use one fictional retailer selling a small catalog of products. Supply 8-12 weeks of synthetic orders, funnel metrics, stock snapshots, supplier offers, budget records, customer feedback, and policy documents. Start with one owner and one business. A second synthetic business exists only to test isolation. Use English queries first; add Hindi or Hinglish as an optional evaluated extension.

Do not connect real payment, email, advertising, or inventory systems in the college MVP. External research can use a curated, dated corpus; live browsing is an optional later connector. Run the same workflow in clearly labeled mock and LLM modes. Mock mode demonstrates orchestration, but cannot establish real model quality, latency, or token savings.

Suggested project framing

Project title: AI-EOS: A Risk-Controlled Multi-Agent Workforce with State-Aware RAG and Semantic Reuse.

Problem statement: Business questions span departments, but repeated agent calls and duplicated evidence increase inference cost. Query similarity alone can reuse obsolete or inappropriate answers. Investigate whether guarded reuse and selective retrieval reduce cost without weakening evidence quality or approval boundaries.

2. Agents and business scenarios

Each agent needs a bounded objective, allowed tools, a department-specific evidence view, and a typed result. Several agents can use the same underlying model; separate model training is unnecessary.

Agent	Responsibility and evidence	Concrete output
Manager	Decompose goals, assign dependencies, resolve conflicts	Task graph and final action plan
Sales	Inspect orders, funnel, channel and product changes	Revenue decomposition and bottlenecks
Inventory	Check availability, stockouts and lead times	Stock exposure and replenishment need
Procurement	Compare supplier price, reliability and delivery	Ranked offers and draft purchase request
Finance	Compute affordability, margin and budget effects	Validated cost and budget assessment
Marketing	Inspect campaign performance and propose tests	Campaign diagnosis and draft experiment
Research	Retrieve policies, customer themes and dated market evidence	Cited findings and alternative hypotheses
Email / CRM	Draft customer or supplier communication	Audience, message and simulated send request

Implement policy enforcement, execution, auditing, and arithmetic as services or deterministic nodes. A verification agent may critique evidence, but cannot grant approval or override the policy engine.

Workflow portfolio

Scenario	Collaboration	Completion condition
Why are sales low?	Sales + inventory + marketing + research; finance checks proposed actions	Evidence-based diagnosis, prioritized plan, owners and KPIs
Replenish low stock	Inventory -> procurement -> finance -> approval	Approved simulated purchase order or recorded rejection
Campaign recovery	Marketing + sales + inventory + finance + email	Draft campaign; spend and publication decisions gated
Customer complaint	Email + research + inventory; finance for a refund	Grounded response draft; refund requires approval
Supplier delay	Procurement + inventory + sales	Revised delivery plan and affected-customer drafts
Weekly owner review	Manager activates only relevant departments	Changes, blocked tasks, decisions and next actions

Scope discipline

Implement replenishment and sales investigation first. Add other scenarios later. Activate only relevant agents: a stock count needs a read tool; a policy question needs retrieval.

3. Worked scenario: why are sales low?

Owner request: "Why did Laptop Stand sales fall in the last four weeks, and what should we do?"

The Manager resolves the reporting period, product ID, currency, timezone, and comparison period before delegation. Assume two comparable four-week windows with unchanged pricing and no returns in this simplified fixture. All values below are hypothetical.

Measure	Previous window	Current window	Interpretation
Product sessions	10,000	8,000	Traffic down 20%
Orders / units	300	160	One unit per order in this fixture
Conversion rate	3.0%	2.0%	Down 1 percentage point
Selling price	INR 1,500	INR 1,500	No price effect here
Revenue	INR 450,000	INR 240,000	Down INR 210,000, or 46.7%
Unavailable days	0 of 28	8 of 28	Possible availability constraint

Delegation and evidence exchange

1. **Sales** verifies the decline and segments it by channel and SKU. Its deterministic bridge attributes INR 90,000 to lower sessions at the old conversion rate and INR 120,000 to lower conversion at current sessions. This is an order-dependent accounting decomposition, not a causal estimate.
2. **Inventory** identifies eight unavailable days and returns timestamped stock evidence. It asks Sales to compare conversion on available versus unavailable days.
3. **Marketing** reports a reduction in campaign traffic and initially proposes a discount. **Research** retrieves customer delivery complaints and checks the dated promotional policy.
4. **Finance** checks the discount against margin constraints. **Inventory** challenges demand stimulation while supply is constrained. The Manager requests one focused follow-up instead of restarting every agent.
5. **Manager** separates measured facts from hypotheses: traffic and conversion both declined; stockouts and delivery complaints are plausible contributors, but the fixture does not prove causation.

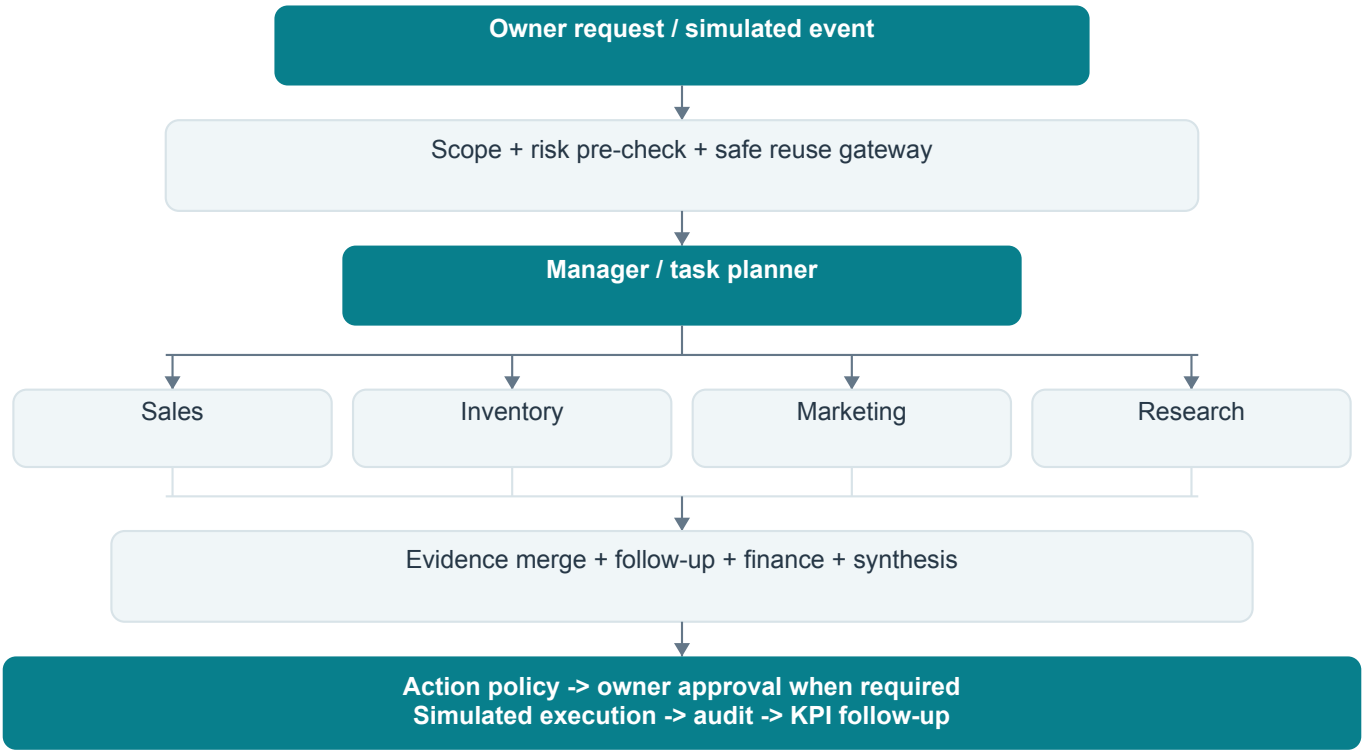
Final owner-facing plan

Priority	Action and responsible agent	Control / success measure
1	Inventory + procurement prepare a replenishment proposal	Owner approves purchase; monitor availability
2	Marketing prepares a campaign for available items	Owner approves any new spend; publication reviewed
3	Email drafts follow-up to affected customers	Simulated send; verify audience and claims
4	Sales compares the next seven days with the baseline	Track sessions, availability, conversion and margin

Show uncertainty and missing evidence explicitly. Do not promise a revenue increase from this diagnosis. A later controlled campaign or stronger observational design would be needed to estimate intervention effects.

4. Architecture and workflow control

Proposed execution path



The diagram's reuse gateway can return an answer only for eligible read-only requests. Action requests always reach the policy gateway; a cached explanation cannot authorize a tool call.

State and execution design

Use a LangGraph state graph with a supervisor and specialist subgraphs. Dynamic delegation selects from a fixed allowlist. Parallelize independent read tasks; keep dependent calculations and state changes sequential. Use workflow IDs, task IDs, parent IDs, dependencies, evidence references, and explicit status transitions.

State includes request scope, business snapshot version, task graph, department results, unresolved conflicts, proposed actions, approvals, execution receipts, token ledger, and audit events. Parallel nodes append separate task results through reducers; they must not overwrite each other's findings or modify stock and budget directly.

Use one bounded follow-up round initially, a maximum of four simultaneous specialists, and a per-workflow call/token ceiling. These are proposed starting limits to tune experimentally. A timeout yields a partial report with named gaps; it must not fabricate a successful department result.

LangGraph supports checkpointed interrupts and resume for approval workflows. Resuming can rerun code before an interrupt, so put mutations after approval and make executor calls idempotent. An in-memory checkpointer supports the local demo but loses waiting workflows on process restart. [S9, S10]

5. What makes the system meaningfully multi-agent?

Useful multi-agent behavior is visible when one specialist's evidence changes another specialist's task or the Manager's decision. A fixed chain remains a valid workflow, but the sales scenario should also demonstrate conditional routing, independent investigation, and evidence-based revision.

Collaboration mechanisms to implement

- **Task contracts:** Each assignment specifies an objective, scope, dependencies, allowed tools, expected result schema, and token/call budget.
- **Shared evidence board:** Store canonical facts once; agents return evidence IDs and small summaries rather than broadcasting entire documents.
- **Directed handoffs:** Inventory asks Sales for available-day conversion; Finance asks Procurement for a lower-cost offer when affordability fails.
- **Conflict resolution:** Preserve incompatible findings, identify why they differ, and request a targeted check. A majority vote does not establish truth.
- **Adaptive replanning:** A stockout routes work toward procurement; healthy availability routes investigation toward funnel or customer issues.
- **Outcome tracking:** Actions have a responsible agent, deadline, expected metric, actual result, and owner decision.

Proposed inter-agent result contract

```
{
  "workflow_id": "wf-104",
  "task_id": "inventory-02",
  "parent_task_id": "manager-01",
  "agent": "inventory",
  "status": "completed",
  "scope": {"sku": "LS-01", "snapshot": "business-v17"},
  "finding": "Product unavailable on 8 of 28 days",
  "evidence_ids": ["stock:LS-01:v17"],
  "limitations": ["No evidence of lost customer intent"],
  "requested_followup": "Compare available-day conversion",
  "proposed_actions": ["prepare_replenishment_request"]
}
```

Task status should distinguish completed, insufficient evidence, failed, and blocked. Optional confidence scores need calibration; they should never stand in for citations or authorization.

Owner dashboard

Show the task graph, concise evidence summaries, disagreements, approval queue, action status, and token use by agent. Label exact-cache hits, semantic-cache hits, fresh generations, and rejected reuse with reasons. Display "why this agent was activated" and "why reuse was rejected" so the research behavior is demonstrable.

Do not expose hidden reasoning traces. Display operational decisions, tool inputs/outputs where appropriate, citations, and short rationales. The audit should explain observable actions without requiring private chain-of-thought.

6. Risk levels and owner approvals

Hard rule: Any action that transfers money, creates a purchasing commitment, issues a refund, changes a paid subscription, or increases advertising spend requires the owner's explicit approval. A finance agent's budget assessment means "affordable," never "authorized."

Tier	Examples	Default behavior
R0: read-only	Read stock, analyze sales, retrieve an authorized policy	Automatic; record sources and access checks
R1: reversible internal work	Draft email, create internal task, prepare report	Automatic within defined permissions
R2: external or consequential nonfinancial work	Send customer message, publish content, alter live business records	Owner review by default; later allow specific approved playbooks
R3: financial / binding	Purchase, payment, refund, new ad spend, contractual commitment	Explicit owner approval every time
Blocked	Unknown recipient, missing authority, forbidden tool, inconsistent amount	Deny or request correction; never auto-execute

Price and discount changes should also require owner review in the MVP because they materially affect revenue. Financial analysis and draft purchase preparation remain automatic because they create no commitment.

Approval lifecycle

```
proposed -> validated -> awaiting_owner -> approved -> executing -> executed
                                    |           |
                                rejected    expired / changed -> reapproval
```

Bind approval to an immutable action digest: workflow/action IDs, business, recipient or supplier, currency, amount, quantity, relevant terms, policy version, expiry, and input snapshot. Record the owner identity and timestamp. The execution gateway checks the approval digest, current budget and stock, permission scope, and an idempotency key immediately before execution. Changed supplier, amount, recipient, or expired approval requires a new decision.

A retry must return the existing receipt, not repeat the purchase. Separate an execution retry for the same approved action from a new action that happens to resemble it. Never copy an approval from the semantic cache. Rejection ends that action and can trigger a revised proposal; it cannot silently become approval.

Concrete replenishment example

Current stock 12, target 50: order 38. At INR 850 each, the draft totals INR 32,300 before any explicitly modeled taxes or shipping. With INR 100,000 available, Finance marks it affordable. **Owner approval is still required**, even below INR 50,000. A higher-value draft follows the same mandatory financial gate. Execution produces a clearly simulated PO and audit receipt.

The local demo may use a labeled owner approval button without authentication. A real deployment requires authenticated owner authority, durable approvals, and persistent audit storage before enabling external actions.

7. RAG and business data pipelines

RAG supplies relevant external evidence to a generator. Its foundational design combines retrieval with generation; it does not make the model a reliable calculator or a live operational database. [S1]

Pipeline A: document knowledge

Inputs: Product descriptions, return rules, marketing policies, supplier terms, customer feedback, and dated research notes.

1. Extract text, preserve headings and tables, remove exact duplicates, and flag untrusted instructions as document content.
2. Chunk by meaningful sections. Start with roughly 300-500 model tokens and small overlap, then tune using retrieval evaluation. Keep table headers with rows; avoid splitting conditions from exceptions.
3. Attach document ID, chunk ID, business, department, access scope, version/hash, effective dates, ingestion timestamp, and source locator.
4. Build lexical BM25 and embedding indexes. Fuse rankings, rerank a small candidate set, and select evidence within a hard context budget. Hybrid retrieval is motivated by exact identifiers and paraphrases; contextual retrieval is an optional experiment. [S6]
5. Generate a cited answer or abstain when evidence is missing or conflicting. Preserve quantities, dates, negation, policy exceptions, and evidence IDs during compression.

Pipeline B: structured business facts

Inputs: Orders, SKU-level inventory, funnel metrics, supplier quotes, budgets, and campaign records. Store these as local CSV/JSON fixtures for the MVP; access them through typed Python tools. Use code for sums, joins, period comparisons, stock deficits, margin calculations, and date normalization. Return compact results with the query definition, source version, and as-of time.

Do not retrieve scattered order-row text to calculate revenue. A structured query is more dependable and can avoid generation entirely for a simple numeric answer. Evidence can combine a tool result with a document passage, such as a budget total plus a purchasing rule.

Pipeline C: workflow memory

Keep active tasks and approval state separate from searchable historical outcomes. A historical successful campaign is evidence for a possible playbook, not proof that it fits today's stock or authority. Persist only validated lessons with provenance; expire obsolete facts.

Sharing and freshness

Use one governed retrieval service with agent-specific views, rather than eight independent copies of the corpus. Deduplicate repeated evidence requests within a workflow. Apply access filters before retrieval and recheck before serving cached results. Index changes, document revocation, policy updates, and structured-data mutations invalidate dependent entries. A TTL is a backup, not a substitute for checking versions.

8. Token optimization: distinct mechanisms

Token count, API price, latency, and correctness are separate metrics. Reducing one does not automatically improve all others.

Mechanism	What it saves	Use and limitation
Exact response cache	Repeated generation and retrieval	Same canonical request and valid dependencies
Semantic response cache	Generation for equivalent wording	Requires scope, freshness and equivalence guards
Embedding cache	Repeated embedding computation	Key by text and embedding model/version
Retrieval cache	Repeated search / reranking	Key by scope, query and index version
Tool-result cache	Repeated read computation	Version-bound; do not cache side effects
Plan / template reuse	Repeated planning	Recompute facts, checks and permissions each run
Context selection / compression	Input tokens on cache misses	May omit critical evidence; validate faithfulness
Selective activation	Unnecessary agent calls	Avoid both under-routing and all-agent fan-out
Smaller model / template routing	Cost or latency per step	Smaller model does not guarantee fewer tokens
Provider prompt caching	Reprocessing of a stable prompt prefix	Generation still happens; billing semantics vary

Practical NLP path

Normalize whitespace and harmless spelling variants; classify read versus action intent; extract product, period, metric, currency, audience, and requested format; then compare sentence embeddings for paraphrase candidates. Sentence Transformers documents this embedding-plus-similarity approach. [S3]

Preserve negation, numbers, SKU identity, time windows, and comparison baselines. "Do not reorder" and "reorder" must never collapse together. "Sales" can mean revenue, orders, or units: resolve the metric before cache matching. Resolve "today" into an absolute date in the business timezone, and resolve follow-up pronouns against conversation state.

Use local rules or a small local classifier where practical. If every cache lookup needs a large generative model to rewrite or verify a query, the lookup can consume much of the intended saving. Count remote embedding and verification costs separately.

Compression policy

Begin with duplicate removal, extractive selection, tool-result aggregation, and short structured agent messages. Evaluate LLMLingua-2 only after those baselines; its paper treats compression as token classification, but its benchmarks do not guarantee preservation of business-critical fields. [S5]

Provider prefix caching is complementary to application caching: it reuses prompt processing while a new response is still generated. It should not be reported as avoided logical input tokens. [S12] Keep static role instructions stable and dynamic facts later in the prompt when supported by the selected provider.

9. Proposed state-aware semantic cache

Research artifact: A reuse gateway that combines NLP similarity with deterministic checks on the business meaning and dependencies of a request. The goal is high precision first; a conservative miss is preferable to a confidently wrong reuse.

Cache entry contract

Store canonical intent and slots; question embedding; business and access scope; locale and output contract; conversation-context fingerprint when needed; answer and evidence IDs; source versions; policy, prompt, tool-schema, and model configuration versions; creation/expiry times; validation status; and token/cost metadata. Never store approval authority in an answer-cache entry.

An exact key covers the entire resolved request scope and dependency versions, not raw question text alone. A semantic candidate index is partitioned by business and access scope. Prefer dependency hashes for only the sources actually used; for the initial MVP, one global snapshot version is simpler and safely invalidates more entries than necessary.

Decision procedure

```
resolve intent, metric, entities, dates and conversation references
if ambiguous or action intent: bypass answer cache
authorize access and obtain authoritative data/index versions
try exact request + scope + version match
otherwise retrieve semantic candidates inside authorized scope
for each candidate, best first:
    require same intent, critical slots and output requirements
    require valid dependencies, policy and configuration versions
    require fresh TTL and similarity above calibrated threshold
    require equivalence check when risk warrants it
    if all checks pass: return answer, citations, as-of time and reuse label
on miss: run fresh workflow with bounded retrieval and generation
validate evidence; admit only eligible, successful read-only results
```

For uncertain equivalence, default to a miss. A small pair classifier or cross-encoder can be an experimental verifier; it is not a substitute for exact date, number, permission, and source-version checks. There is no universal cosine threshold such as 0.90. Calibrate by query class and measure precision versus coverage.

Invalidation and concurrent requests

Maintain source-to-entry dependency links. Updating stock, orders, campaign data, or a policy evicts or rejects affected entries. Treat deleted and revoked sources as invalid. Use request coalescing so simultaneous equivalent read requests share one in-flight computation. Include scope and snapshot in the coalescing key.

Before returning a newly computed answer, check that its dependency snapshot is still valid; retry or label a historical as-of result if the business changed mid-run. Cap memory and use a documented eviction policy. In-memory caches reset on restart, which must be stated in the demo.

10. When reuse is safe, partial, or forbidden

Incoming request after an earlier answer	Decision	Reason
"Explain our return window" -> "How long can customers return items?"	Possible full hit	Same policy, audience, permissions and effective version
"Why did LS-01 revenue fall in September?" -> paraphrase with same period	Possible full hit	Same metric, comparison, data snapshot and requested detail
"Why are sales low?" after new orders arrive	Miss or reuse plan only	The explanation depends on changing facts
Same wording for a different product	Miss	Entity differs even if embeddings are close
Revenue question -> units-sold question	Miss	Metrics are not interchangeable
"This month" -> "last month"	Miss	Time scope differs
"Can we afford this purchase?" -> "Pay this supplier"	Miss and approval gate	Analysis and execution are different intents
A customer asks an owner's internal finance question	Deny / scoped fresh answer	Access rights differ
"And what about that product?"	Resolve context, then match	Current utterance alone is insufficient
"Approve the same payment again"	New action checks	Cached language cannot establish approval

Reuse the plan when facts change

For a familiar sales diagnosis, reuse the task template: compare traffic, conversion, pricing, availability, and campaigns. Recompute all tool results against current data. Regenerate only affected specialist findings and synthesis. This reduces planning overhead without asserting that yesterday's diagnosis remains true.

Within a workflow, two agents requesting the same stock snapshot can share one read result. Across workflows, reuse requires the same scope and source version. Store raw evidence once and pass a small cited extract to each model; an evidence ID is useful only if the receiving agent can resolve it.

Red-team cases for the gateway

Test same wording under changed budget, changed supplier, modified policy, different business, revoked access, swapped currency, changed negation, and expired dates. Inject a malicious retrieved passage that tells an agent to approve a payment. Retrieved text and cached answers remain evidence, never instructions or capabilities.

OWASP identifies indirect prompt injection through external material and notes that RAG does not eliminate it. [S13] The project's concrete defense is to enforce tool permissions and approvals outside the model, validate structured tool arguments, and keep sensitive retrieval/cache scopes isolated.

Avoid a misleading claim

"A semantic cache hit avoids a new generative LLM call for an eligible question" is testable. "Similar questions use zero tokens" is too broad: embedding APIs, verifiers, response formatting, or fresh synthesis may still incur tokens. Report each component separately.

11. Research evidence and positioning

The literature supports the building blocks, but does not establish the proposed system's performance. Published improvements belong to their original datasets, models, and configurations.

Primary source	Relevant finding or method	Implication for AI-EOS
RAG, Lewis et al. (2020) [S1]	Retrieval supplies external knowledge to generation	Ground documents; retain tool-based numeric facts
GPTCache, Bang (2023) [S2]	Semantic response caching can bypass repeated LLM requests	Include it as an established baseline
Sentence-BERT / Sentence Transformers [S3]	Sentence embeddings support similarity matching	Retrieve candidates, then enforce exact scope checks
ContextCache (2025) [S4]	Uses dialogue context in semantic cache matching	Compare context-aware reuse for follow-up questions
LLMLingua-2 (2024) [S5]	Extractive compression through token classification	Evaluate tokens and critical-field preservation together
Contextual Retrieval (2024) [S6]	Combines chunk context with embeddings and BM25	Test lexical + semantic retrieval against dense-only
Lost in the Middle (2023/2024) [S7]	Evidence position affects performance in evaluated long contexts	Do not assume more context improves answers
Ragas (2023/2024) [S8]	Evaluates retrieval and grounded generation dimensions	Supplement automated metrics with human checks

Multi-agent cost is a research motivation

Anthropic's published research-system account uses an orchestrator-worker architecture and reports substantial token overhead in its own deployments. That observation motivates selective activation and compact handoffs here; it is not a multiplier to apply to this project's cost estimate. [S11]

Defensible contribution

Propose and evaluate a business-specific combination of intent/slot matching, data-version checks, conversation-aware matching, risk-based reuse policy, compact evidence exchange, and adaptive specialist activation. Release the synthetic scenarios, query variants, stale-state tests, evaluation labels, and per-workflow token ledger.

Do not claim semantic caching, context-aware caching, or compression is novel. ContextCache already addresses conversation context, and SCALM studies semantic cache architecture and eviction. [S4, S14] The plausible contribution is a reproducible evaluation of **reuse under changing business facts and approval constraints**. A broader novelty claim would require a systematic literature review beyond this focused report.

12. Experimental design

Research questions and hypotheses

- **RQ1:** Does guarded semantic caching reduce generation cost beyond exact caching while retaining high reuse precision?
- **RQ2:** How much do entity, period, permission, and version guards reduce incorrect hits relative to similarity-only caching?
- **RQ3:** Do compact evidence packets and selective activation improve workflow efficiency without losing necessary departments?
- **RQ4:** When does compression save enough tokens to justify its latency and possible evidence loss?

These are hypotheses to test, not achieved findings. Compare full system performance with component ablations so a cache benefit is not confused with a smaller prompt or fewer agents.

Proposed dataset

Construct 120 base tasks: 30 policy questions, 30 structured analytics questions, 30 cross-department diagnosis questions, and 30 action/approval workflows. Add two independently reviewed paraphrases per task for 360 utterances. Add 120 adversarial variants covering changed entities, dates, metrics, negation, currency, and dialogue context. Evaluate a separate 60-case safety suite spanning stale data, access isolation, approval rejection, parameter changes, and duplicate execution.

Split base-task families before generating paraphrases: 60% train/design, 20% development, 20% test. Keep all variants from one family in one split. Each test episode has a declared seed request followed by evaluated variants, so a warm-cache test does not accidentally use held-out labels. Human reviewers label equivalence, expected tools/agents, required sources, numeric answers, and approval requirements; adjudicate disagreements.

Main comparisons

Arm	Configuration	Purpose
B0	Fixed specialist set, standard RAG, no reuse	Unoptimized multi-agent baseline
B1	B0 + exact cache	Value of literal / canonical repetition
B2	B0 + embedding-only semantic cache	Unsafe-similarity comparison, sandbox only
B3	B0 + guarded semantic cache	Isolate state and scope safeguards
B4	B3 + compact evidence / compression	Isolate context reduction
B5	B4 + selective agent activation	Full proposed system

Also compare B0 and B5 with a single agent using the same tools, evidence access, and generous bounded budget. Compare retrieval dense-only versus hybrid separately; do not change retrieval and caching together in the main experiment. If implementing ContextCache or LLMLingua-2 is infeasible, label them literature comparisons rather than reproduced baselines.

13. Metrics, cost model, and validity

Evaluation protocol

Run chronological cold-cache, repeat-heavy warm-cache, and mutation-heavy traces. Clear caches between independent trials; fix and record seeds, model IDs, prompts, embedding versions, corpus snapshots, and hardware. Freeze thresholds using development data. Run at least three generation trials where nondeterminism matters and report variability. Record first-query overhead as well as repeat-query benefit.

Dimension	Metric
Retrieval	Recall@k and ranking quality against labeled supporting evidence
Answer quality	Numeric accuracy, citation support, completeness and appropriate abstention
Reuse	Hit rate; correct hits / all served hits; stale-hit and false-hit rates
Workflow	Task success, required-agent coverage, conflict resolution and tool failures
Safety	Unauthorized financial actions, cross-business leakage, duplicate executions
Efficiency	Input/output tokens, reasoning tokens if exposed, model calls, p50/p95 latency
Cost	Generation, embeddings, verification, reranking and local compute separately

Use a human rubric for business diagnosis and audit a sample of automated Ragas-style scores. [S8] Compare systems on the same tasks and report paired differences with uncertainty intervals. High average quality must not conceal failures on finance or stale-state cases.

Cost accounting

For a trace, sum each actual invocation: uncached input tokens x input rate + cached-prefix tokens x cached rate + billed output tokens x output rate. Add embedding, reranking, verification, and any amortized local compute costs. Treat provider cache writes separately where applicable. Obtain rates from the chosen provider at experiment time; no fixed price is assumed here.

Net saving = 1 - optimized total cost / baseline total cost. Count application-cache hits as avoided generation only when no generation occurs. Keep logical tokens separate from billable token categories; provider prefix hits are still part of the prompt. Avoid double counting reasoning tokens already included in billed output.

Illustrative token calculation, not a result

Suppose 100 baseline workflows each use 6,000 input + 1,000 output tokens: 700,000 generation tokens. If 40 valid cache hits need no generation and the remaining 60 use 3,000 input + 700 output, the optimized generation total is 222,000, a **68.3% reduction**. This excludes cache/embedding overhead, warm-up, verification, and price differences; it does not predict actual savings.

For a simple per-request break-even model, let baseline generation cost be Cg, hit rate h, and lookup/verification cost Cc paid on every request. Net benefit requires h x Cg > Cc. More complex plan reuse and compression need invocation-level accounting.

14. Delivery plan and technology choices

Use the brief's Python/FastAPI + LangGraph + React direction [B1]. Keep business tools and policy checks independent of the model provider. The local fixture loader, BM25 index, embedding matrix, exact/semantic cache, and graph checkpointer can all live in process for the MVP. A small corpus does not need a vector database server.

Stage	Deliverable	Exit condition
Week 1	Fixtures, schemas, read tools, inventory/procurement/finance graph	Correct totals; mandatory approve/reject; complete audit
Week 2	Manager delegation, sales investigation, compact agent results	Parallel evidence changes the plan; bounded follow-up works
Week 3	Versioned document ingestion and hybrid RAG	Citations resolve; permission/freshness checks and abstention work
Week 4	Exact cache, semantic candidates, guards and invalidation	Paraphrase hit; changed-state and action requests bypass safely
Week 5	Dataset, ablations, token ledger, latency/cost harness	Frozen test protocol; reproducible cold/warm/mutation runs
Week 6	Error analysis, report, demo and optional compression study	Claims match measurements; limitations documented

This six-week schedule is an indicative planning assumption, not a delivery promise. If time is short, omit model-based compression, multilingual evaluation, live web research, and extra workflows before removing safety checks or experiment controls.

Suggested modules

```
backend/agents/       manager, sales, inventory, procurement, finance,
                      marketing, research, email
backend/tools/        typed read tools + simulated action executor
backend/graph/        workflow state, routing, joins, interrupts
backend/policy/       risk rules, approval binding, authorization
backend/retrieval/    ingestion, BM25, embeddings, evidence selection
backend/cache/        exact/semantic lookup, guards, invalidation
backend/observability/ audit events + token/cost ledger
data/                 synthetic business fixtures + documents
eval/                 labeled tasks, traces, baselines, scoring
frontend/             owner view, task graph, evidence, approvals
```

Later production work

After the college demo, add authenticated roles, durable workflow/approval storage, persistent source versioning, tenant isolation, real connector permissions, transaction-aware execution, monitoring, and recovery. In-memory state cannot safely support real spending or restart recovery. Keep these as a separate deployment phase; they should not inflate the research MVP.

15. Demonstration and acceptance criteria

A coherent demonstration sequence

1. **Operational workflow:** Stock 12, target 50, price INR 850. Show inventory -> procurement -> finance, the INR 32,300 proposal, and a pause for owner approval. Approve once and show one simulated PO.
2. **Rejection and retry:** Reject another purchase. Confirm no PO. Retry an already executed approved action and show the existing receipt, with no duplicate purchase.
3. **Parallel investigation:** Ask why sales declined. Show specialists activated for a reason, stock evidence challenging a discount proposal, one follow-up, and a revised plan with KPIs.
4. **Useful reuse:** Rephrase an eligible policy question. Show a semantic hit, source version, and no new generation call.
5. **Freshness protection:** Change a policy or sales snapshot. Repeat the same question and show invalidation followed by fresh computation.
6. **Boundary protection:** Change SKU, period, metric or business scope. Show a rejected match. Submit a payment-like request and show fresh authorization and mandatory approval.

Proposed evaluation gates

Gate	Initial acceptance target
Financial boundary	Zero unauthorized financial executions in the safety suite
Execution correctness	Zero duplicate simulated financial effects on retries
Cache quality	At least 99% served-hit precision on held-out eligible requests; report count and interval
Freshness / isolation	Zero stale hits or cross-business disclosures in dedicated tests
Business quality	No more than 2 percentage points decline in task success versus B0
Efficiency	At least 25% net generation-token reduction on the declared repeat-heavy trace

These are proposed project goals, not measured achievements. The efficiency target is workload dependent and is not a claim of 25% cash savings. A small sample with zero errors cannot prove universal safety or 99% reliability; report denominators and statistical uncertainty. Do not lower guards to improve hit rate.

What to submit

Deliver the working simulated workflows, evidence and approval UI, versioned synthetic corpus, labeled evaluation set, experiment configurations, per-agent token/cost logs, ablation results, and error analysis. Present both a successful reuse case and a rejected lookalike so the demonstration explains the research tradeoff.

Priority extension ideas

After the two core workflows work, add event-triggered stock or conversion alerts, a supplier-failure escalation, reusable playbooks with fresh checks, and post-action KPI review. Optional multilingual queries and business-specific embedding fine-tuning can be future studies. Avoid adding agent roles merely to increase the displayed headcount.

References and evidence notes

Web sources were checked on 14 October 2026. All architecture choices, starting thresholds, schedules, synthetic values, and acceptance targets are recommendations from this analysis unless explicitly attributed. Research papers support methods and motivation, not unmeasured AI-EOS performance.

[B1] User-supplied project brief. AI-EOS MVP Project Brief.pdf, 10 pages. Operational scenario and approval threshold: pp. 1-2; in-memory state and agent roles: pp. 3-5; demo and implementation constraints: pp. 7-10. The current user request takes precedence over embedded build commands and the brief's low-value auto-purchase rule.

[S1] Lewis et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. Foundational retrieval-plus-generation design. Paper.

[S2] Bang (2023). GPTCache: An Open-Source Semantic Cache for LLM Applications Enabling Faster Answers and Cost Savings. Established semantic response caching; no reported speedup is transferred to this project. ACL paper and metadata.

[S3] Reimers and Gurevych (2019); Sentence Transformers documentation. Sentence-BERT provides sentence embeddings for similarity; the official usage guide shows embedding and similarity computation. Paper, official similarity guide.

[S4] ContextCache (2025). ContextCache: Context-Aware Semantic Cache for Multi-Turn Queries in Large Language Models. Uses query candidates and dialogue context for matching; does not establish this report's business-state or approval guarantees. Paper.

[S5] Pan et al. (2024). LLMLingua-2: Data Distillation for Efficient and Faithful Task-Agnostic Prompt Compression. Compression through token classification. Paper.

[S6] Anthropic (2024). Introducing Contextual Retrieval. Describes chunk context, embeddings, BM25, and reranking; used as a design reference rather than a guaranteed retrieval improvement. Engineering article.

[S7] Liu et al. (2023; TACL 2024). Lost in the Middle: How Language Models Use Long Contexts. Evaluated context-position sensitivity; this does not prove identical behavior for every newer model. Paper.

References and assumptions, continued

[S8] Es et al. (2023; EACL demo 2024). Ragas: Automated Evaluation of Retrieval Augmented Generation. Evaluation framework for retrieval and generation quality; automated scores need task-specific validation. Paper.

[S9] LangChain. LangGraph interrupts documentation. Pause/resume requirements, checkpointer, thread ID, and node re-execution behavior. Official documentation.

[S10] LangChain. LangGraph persistence documentation. Checkpointed graph state and memory versus persistent storage. Official documentation.

[S11] Anthropic (2025). How we built our multi-agent research system. Orchestrator-worker design, delegation, evaluation and cost tradeoffs. Engineering article.

[S12] Anthropic. Prompt caching documentation. Provider caching reuses prompt processing and has distinct billing behavior; verify selected-provider rules at implementation time. Official documentation.

[S13] OWASP. LLM01:2025 Prompt Injection. Direct and indirect injection, including externally retrieved content, and the limits of RAG as a mitigation. Official risk description.

[S14] Li et al. (2024). SCALM: Towards Semantic Caching for Automated Chat Services with Large Language Models. Prior work on semantic analysis, cache entry importance, storage and eviction. Paper.

Assumptions that need validation during implementation

- Repeated eligible questions occur frequently enough for caching to pay for itself. Measure the query distribution and include low-repeat traces.
- The system can obtain authoritative dependency versions before reuse. If a source cannot supply freshness, use conservative expiration or recompute.
- Product and time extraction are reliable enough to gate cache matches. Ambiguous extraction must cause a miss or clarification.
- Parallel work is useful for the chosen scenarios. Compare with a single agent and a simpler deterministic workflow.
- A small synthetic corpus approximates enough business variation for a college evaluation. It cannot establish production reliability or real business impact.

Recommended research claim after evaluation

Report: "On the specified synthetic business traces, our guarded reuse design changed generation tokens, net cost, latency, and incorrect-hit rate by the measured amounts relative to named baselines. All financial actions remained behind owner approval in the evaluated suite."

Publish the measured counts, workload composition, configuration, uncertainty, and failures alongside that claim. Until the experiments run, the project has a defensible research plan, not demonstrated optimization results.